

Department of Computer Science
Ashoka University
Advanced Algorithms
Assignment 1

Collaborators: None

Name: Rushil Gupta

Problem 2

Consider the bichromatic coloring problem: Given a set S of n elements, and subsets $S_1, S_2, \dots, S_k$ of S , find a 2-coloring of the elements in S such that every S_i contains at least one red and at least one green element.

We will analyse the randomized algorithm that independently colors each element of S red or green with probability $1/2$ under the Las Vegas, Monte Carlo and derandomized settings.

Las Vegas

The Las Vegas algorithm will keep randomly coloring the elements of S until a valid coloring is found. We are interested in the expected number of iterations before a valid coloring is found. Since each trial is independent, this problem reduced to finding the success probability of a single trial.

Let A_i be the event that the subset S_i is not bichromatic, i.e. a failure event. Then, $Pr(A_i) = 2^{1-|S_i|}$, since there are two ways in which S_i can be monochromatic (all red or all green) and $2^{|S_i|}$ total ways to color the elements of S_i .

Let A be the event that the coloring is not valid, i.e. at least one of the subsets is not bichromatic. Then, by union bound, we have

$$Pr(A) \leq \sum_{i=1}^k Pr(A_i) = \sum_{i=1}^k 2^{1-|S_i|}$$

Therefore, the probability of success in a single trial is

$$Pr(\text{success}) = Pr(A^c) = 1 - Pr(A) \geq 1 - \sum_{i=1}^k 2^{1-|S_i|}$$

A remark here is that for this to be a useful lower bound on the failure probability, it suffices to have $\sum_{i=1}^k 2^{1-|S_i|} < 1$.

Assuming this is true, the expected number of trials before a success is

$$\mathbb{E}[\# \text{ trials}] = \frac{1}{Pr(\text{success})} \leq \frac{1}{1 - \sum_{i=1}^k 2^{1-|S_i|}}$$

The cost of verifying if a coloring is valid is $O(N)$, where $N = \sum_{i=1}^k |S_i|$. Therefore, an upper bound on the expected running time of the Las Vegas algorithm is:

$$O\left(\frac{N}{1 - \sum_{i=1}^k 2^{1-|S_i|}}\right)$$

Monte Carlo

The same argument as above shows that the probability of failure in a single trial is at most $\sum_{i=1}^k 2^{1-|S_i|}$. Therefore, if we run the algorithm for t trials, the probability of failure is at most:

$$Pr(\text{fail in all } t \text{ trials}) \leq \left(\sum_{i=1}^k 2^{1-|S_i|}\right)^t$$

The running time of the Monte Carlo algorithm is $O(Nt)$, where $N = \sum_{i=1}^k |S_i|$. Therefore, for any $\delta > 0$, if we want the probability of failure to be at most δ , we can set

$$t \geq \frac{\log \delta}{\log \left(\sum_{i=1}^k 2^{1-|S_i|} \right)}$$

The running time of the algorithm is independent of the success probability, and is:

$$O(N \cdot t)$$

where t can be chosen based on the desired probability of failure δ .

Derandomized

Consider a graph-like data structure where each element in S is a vertex, and each subset S_i is also a vertex. There is an edge between an element vertex and a subset vertex if the element is in the subset.

This data structure maintains a partial coloring of the elements in S , and for each S_i , we know the number of colored elements of each color (keep a counter on the subset vertices). This also allows us to know all the subsets a given element belongs to.

We will iteratively color the elements of S one by one, and at each step, we find the failure probability condition on coloring the current element red or green, and choose the color that leads to the lower failure probability. The proof of correctness for this given that the initial failure probability is less than 1 was done in the lecture.

By the proof covered in the lecture, knowing which case we lie in (1, 2 or 3). Knowing the number of uncolored elements in each subset, we can compute the failure probability of the current partial coloring conditioning on coloring the current element red or green.

It is important to see that this is a constant amount of work for each subset. So, at each step, the work done is proportional to the number of subsets the current element belongs to. This gives us the time complexity of the derandomized algorithm as:

$$O(N) = O \left(\sum_{i=1}^n \# \text{ of times element } i \text{ appears in subsets} \right) = O \left(\sum_{i=1}^k |S_i| \right)$$

Remark: This discussion and analysis of the problem and 3 solutions did not need the subsets to be of the same size. The analysis holds for arbitrary subset sizes.

Problem 3

Consider the 0–1 Knapsack problem: we have a capacity C and items with weight w_i and profit p_i . We look at a simple greedy algorithm based on the profit density p_i/w_i . Sort all items in non-increasing order of p_i/w_i : $x_1, x_2, \dots$. Take items in this order while they fit. Suppose we successfully take $x_1, \dots, x_k$, but the next item x_{k+1} does not fit. Let

$$G_1 = \sum_{i=1}^k p_i \quad \text{and} \quad G_2 = p_{k+1}.$$

We output

$$G = \max\{G_1, G_2\}.$$

Claim: This value G is at least half of the optimal 0–1 knapsack profit.

Consider the fractional version of knapsack where we are allowed to take a fraction of one item. The greedy-by-density method is optimal for the fractional problem: it takes all of $x_1, \dots, x_k$ and then as much as possible of x_{k+1} . Let OPT be the optimal 0-1 knapsack profit and let OPT_{frac} be the fractional profit of this greedy solution. We have $\text{OPT} \leq \text{OPT}_{\text{frac}}$ because allowing fractions cannot reduce the profit.

The fractional value is

$$\text{OPT}_{\text{frac}} = \sum_{i=1}^k p_i + \left(C - \sum_{i=1}^k w_i\right) \cdot \frac{p_{k+1}}{w_{k+1}}.$$

Since x_{k+1} did not fit completely, we have $C - \sum_{i=1}^k w_i < w_{k+1}$. Therefore

$$\text{OPT}_{\text{frac}} < \sum_{i=1}^k p_i + p_{k+1} = G_1 + G_2.$$

Combining gives

$$\text{OPT} \leq \text{OPT}_{\text{frac}} < G_1 + G_2.$$

Thus at least one of G_1 or G_2 is at least half of OPT , i.e.

$$\max\{G_1, G_2\} \geq \frac{1}{2} \text{OPT}.$$

This is precisely saying that this modified greedy algorithm gives at least half of the optimal profit.

Problem 10

We have a bin with n balls, where n is even. Exactly $n/2$ are red and $n/2$ are black. We draw balls one by one until we get a red ball. We compare two schemes: i) with replacement, and ii) without replacement.

Let T be the draw number on which we first see a red ball.

With replacement

Each draw is an independent Bernoulli trial with success probability $p = 1/2$ (success = red). Therefore

$$Pr(T = k) = (1 - p)^{k-1}p = (1/2)^k, \quad k = 1, 2, 3, \dots$$

and

$$Pr(T > k) = (1 - p)^k = (1/2)^k.$$

This is the standard geometric distribution. Its mean is

$$\mathbb{E}[T] = \frac{1}{p} = 2$$

Without replacement

Here, the analysis is a bit more tedious, but it is intuitive to think why this should help: after seeing a black ball, there are more red balls than black balls, so the chance of seeing a red ball in the next draw increases.

We will do the following analysis. Consider a sequence of n balls, with $n/2$ red and $n/2$ black, with all such sequences equally likely. Place a red ball at the start and end of the sequence. Now, the number of draws until we see a red ball is the position of the first red ball in this modified sequence. So, we want to know the expected length of the first segment of black balls in this modified sequence.

Now, we have $\frac{n}{2} + 2$ red balls (ends of contiguous segments of black balls) and $\frac{n}{2} + 1$ gaps between them (segments of black balls). Then, the expected length of these gaps are the same due to symmetry and the fact that we are considering all sequences equally likely. Since we have $n + 1$ balls in the sequence (we are not counting the red ball at the start since we think of that as the "0th" draw), the expected length of each gap is:

$$\mathbb{E}[T] = \frac{n + 1}{\frac{n}{2} + 1}$$

Clearly this is less than 2 for all $n \geq 0$, and tends to 2 as $n \rightarrow \infty$. Thus, the expected number of draws until we see a red ball is less when we draw without replacement, as intuition suggested.

Problem 12

We will analyze the space usage and search time of a randomized skip list with n elements where each element is independently promoted to the next higher level with probability $1/2$.

Let $S(n)$ be the total space (number of node copies across all levels) and $T(n)$ be the search time for any fixed element.

Search Time

We want to show that for some constant $c > 0$ and any $\alpha \geq 1$:

$$Pr(T(n) \geq \alpha \cdot c \cdot \log n) \leq \frac{1}{n^\alpha}$$

The search process moves right within levels and down between levels. Let H be the maximum height of the skip list. The total search cost is proportional to H plus the number of horizontal moves, so we will bound both these quantities separately.

The probability that any element reaches height h is 2^{-h+1} , since it is a geometric random variable. By union bound:

$$Pr(H \geq h) \leq n \cdot 2^{-h+1}$$

Setting $h = 2\alpha \log n$ gives:

$$Pr(H \geq 2\alpha \log n) \leq n \cdot 2^{-2\alpha \log n + 1} = \frac{2}{n^{2\alpha}} \leq \frac{1}{n^\alpha}$$

Now, note that at each level, the expected number of horizontal moves is constant (at most 2) due to the geometric distribution of gaps between promoted elements. So:

$$Pr(T(n) \geq c\alpha \log n) \leq \frac{1}{n^\alpha}$$

Space Bound

We want to show that for any $a > 0$:

$$Pr(S(n) \geq 2n + an) \leq 2^{-a^2 n}$$

First, note that the expected space is $\mathbb{E}[S(n)] = 2n$ since each of the n elements has expected height 2. Now, we know for $\delta \in [0, 1]$:

$$Pr(S(n) \geq (1 + \delta) \cdot 2n) \leq e^{-\frac{\delta^2}{3} \cdot 2n}$$

Setting $\delta = a/2$ gives:

$$Pr(S(n) \geq 2n + an) \leq e^{-\frac{a^2}{12} \cdot 2n} = e^{-\frac{a^2}{6} n} \leq 2^{-a^2 n}$$